

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of WEDNESDAY 12 FEBRUARY 2025 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

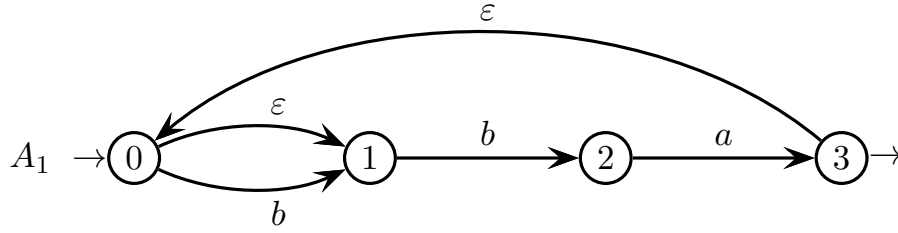
Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

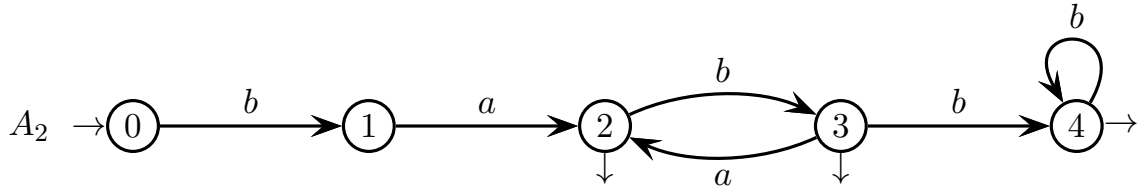
- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Regular expressions and finite automata (10 points)

1. Consider the nondeterministic automaton A_1 below, over the two-letter alphabet $\Sigma = \{a, b\}$:



the automaton deterministic A_2 below, over the same alphabet $\Sigma = \{a, b\}$:



and the regular expression R , also over the alphabet $\Sigma = \{a, b\}$:

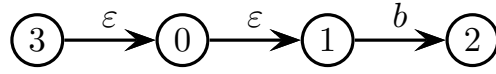
$$R = ab^+ (ab)^* \mid (ba)^+ (ab)^+$$

Answer the following questions (use the figures / tables / spaces on the next pages):

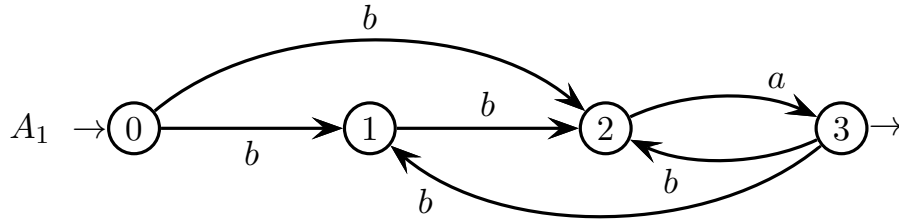
- Apply the backward propagation method on automaton A_1 to produce an automaton A'_1 without epsilon-moves (spontaneous moves).
- Say if the automaton A'_1 , obtained at point (a) by eliminating the epsilon-moves, is deterministic or not. Briefly explain your answer.
- Use the Brzozowski-McCluskey node elimination method (BMC) to produce a regular expression R_{A_2} from automaton A_2 .
- Apply the Berry-Sethi (BS) method to the regular expression R to obtain an equivalent deterministic automaton A_R .
- Say if the automaton A_R obtained through the BS method at point (d) is minimal, and explain your answer. If the automaton is not minimal, minimize it.
- Say if the language described by the intersection $L(A_2) \cap L(R)$ is empty or not. Briefly explain your answer.

Solution

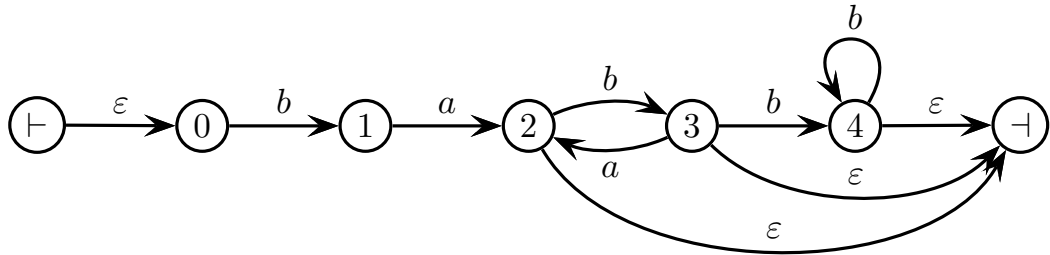
- (a) The elimination of the ε -moves is fairly straightforward, exception made for the path below:



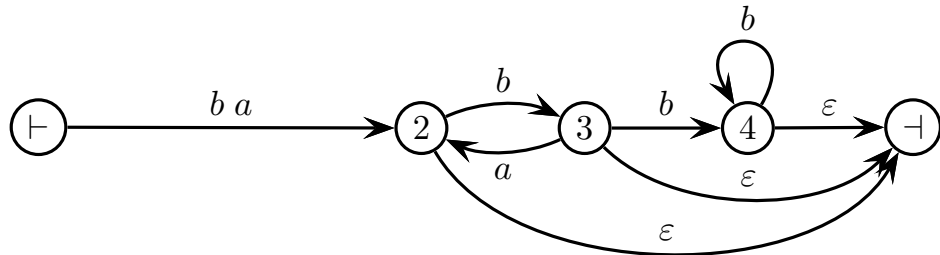
which includes an ε -path of two ε -moves, hence it is easy to overlook. This path generates, by backward propagation, the transition $3 \xrightarrow{b} 2$. There are no ε -moves to final states, therefore there is no need to mark any additional state as final (state 3 remains the only final one). The resulting automaton follows (by backward propagation):



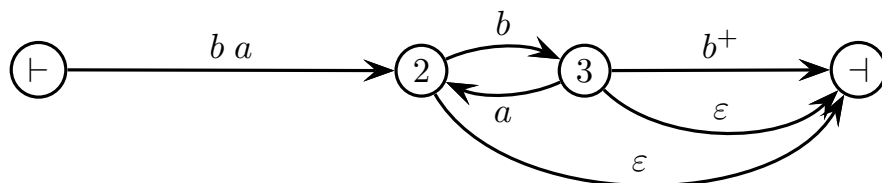
- (b) The automaton A_1 is clearly nondeterministic, as both states 0 and 3 have two outgoing transitions with the same label b .
- (c) First, create unique source ($\vdash$) and sink ($\dashv$) states. Notice there are three final states, namely 2, 3 and 4, which all need to be connected to the sink and made non-final:



Simultaneously eliminate states 0 and 1. They form a straight path from state $\vdash$ to state 2, so eliminating them is straightforward.



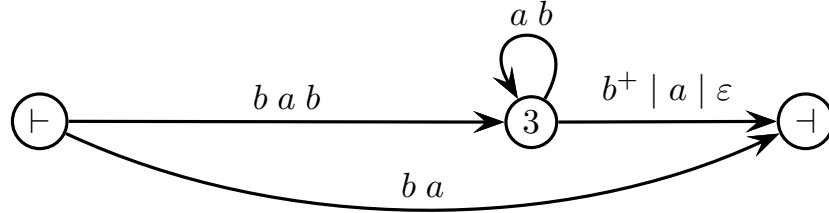
Eliminate state 4. This is another unique path (from state 3 to state $\dashv$) but with a self-loop on state 4; therefore the regular expression is $b b^*$, which can be simplified to b^+ :



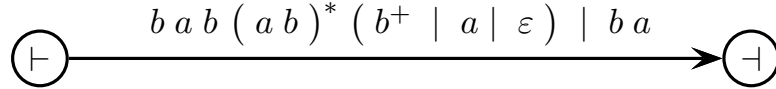
Now we choose to eliminate state 2. There are two incoming paths to state 2 (from states $\vdash$ and 3) and two outgoing paths (to states 3 and $\neg$), so we need to add four transitions to the machine (which replace the ones involving state 2):



We coalesce the last one of these transitions (the one from state 3 to state $\neg$) with the other two parallel ones between the same pair of states, thus making a single regular expression with two union operators:



Finally, we eliminate state 3. This makes two parallel transitions between states $\vdash$ and $\neg$, which generates yet another union operator. Notice the added parentheses, which are needed because the union operator has the lowest precedence:



As an additional comment to the solution, we can observe that with appropriate simplifications this apparently complex regular expression is equivalent to the much simpler one:

$$R'_{A_2} = (b a)^+ b^*$$

which goes to show how many regular expressions can be written to describe the same language. Other formulations could have been written as well, by eliminating the states in a different order. Notably, if we had eliminated state 3 before state 2, we would have obtained:

$$R''_{A_2} = (b a)^+ (b b^+ | b | \epsilon)$$

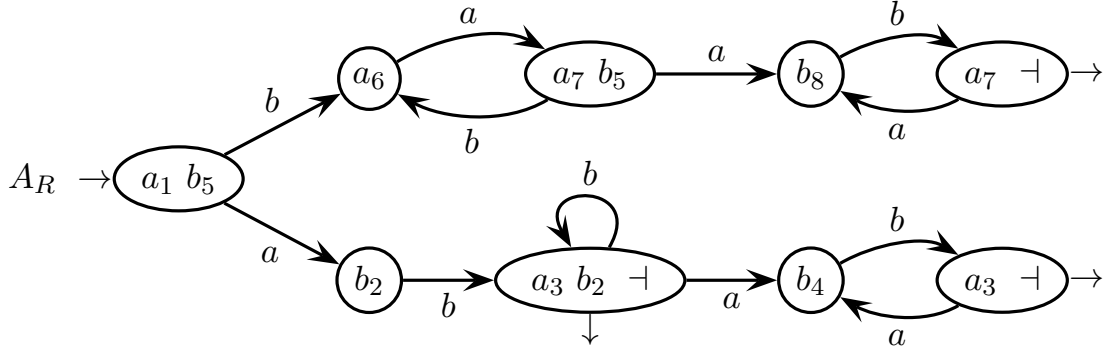
which is even easier to simplify, since clearly $(b b^+ | b | \epsilon) = b^*$.

- (d) Here is Berry-Sethi method applied to the regular expression R . The resulting machine has two non-intersecting execution branches, which originate directly from the initial state, and faithfully reflect the structure of the original regular expression almost as if it was produced by the Thompson algorithm (modular method) instead.

This is due to the fact that there are no ambiguities in the regular expression itself, and hence the sets of followers always have only one instance of each (non-numbered) terminal.

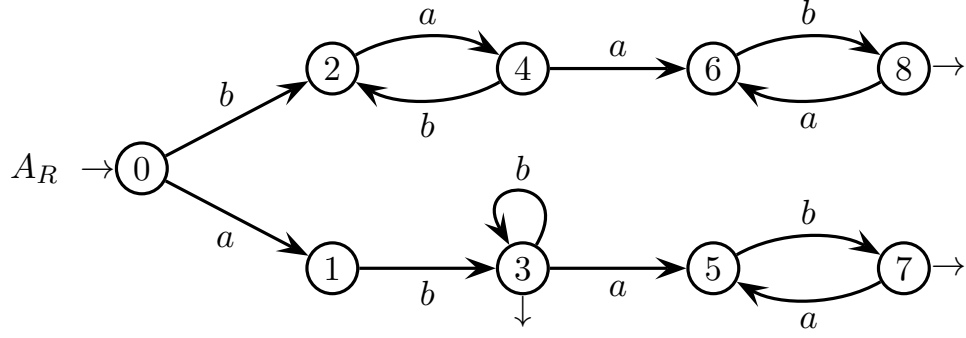
$$R_{\#} = a_1 b_2^+ (a_3 b_4)^* | (b_5 a_6)^+ (a_7 b_8)^+$$

initials	$a_1 b_5$
terminal	followers
a_1	b_2
b_2	$a_3 b_2 \dashv$
a_3	b_4
b_4	$a_3 \dashv$
b_5	a_6
a_6	$a_7 b_5$
a_7	b_8
b_8	$a_7 \dashv$



Automaton A_R is surely deterministic and clean, but it may be non-minimal.

(e) Let us first renumber the states in automaton A_R for ease of exposition:



Then we can proceed – for instance – by applying the Myhill-Nerode minimization method:

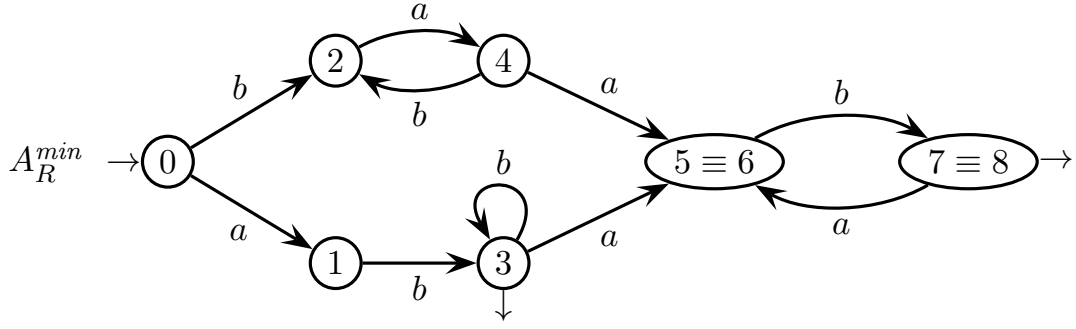
q	final	$\delta(q)$
0		(1 , 2)
1		(- , 3)
2		(4 , -)
3	•	(5 , 3)
4		(6 , -)
5		(- , 7)
6		(- , 8)
7	•	(5 , -)
8	•	(6 , -)

1	(1,2)						
2	(1,2)	(4,3)					
3	(1,2)	(4,3)	(5,3)				
4	(1,2)	(4,3)	(6,2)	(6,2)			
5	(1,2)	(4,3)	(5,3)	(5,3)	(5,2)		
6	(1,2)	(4,3)	(6,2)	(6,2)	(6,2)	(6,2)	(6,2)
7	(1,2)	(4,3)	(5,3)	(5,3)	(5,3)	(5,3)	(5,3)
8	(1,2)	(4,3)	(6,2)	(6,2)	(6,2)	(6,2)	(6,2)

The elimination steps go as follows:

1. The states of the pairs (1,0), (2,0), (5,0), (6,0), (2,1), (4,1), (3,2), (7,3), (8,3), (5,4) and (6,4) are distinguishable because they have different sets of undefined transitions. For instance, state 0 can read an a while state 1 cannot, so they are distinguishable.
2. The states of the following pairs are distinguishable because with at least one label they transition to states that have already been marked distinguishable in step 1:
 - (5,1) because they transition to (7,3) with b
 - (6,1) because they transition to (8,3) with b
 - (7,2) because they transition to (5,4) with a
 - (8,2) because they transition to (6,4) with a
3. The states of the pair (4,0) are distinguishable because with label a they transition to states (6,1), which have already been marked distinguishable in step 2.
4. No other pair of states can be marked distinguishable, and the algorithm stops.

This leaves us with state pairs (6,5) and (8,7), which have not been eliminated. They form two distinct equivalence classes (indeed we have two two-element cliques). The minimized automaton A_R^{min} follows:



Alternatively, rather than carrying out the entire minimization process according to the Myhill-Nerode method, one could simply observe that there are duplicate states and transitions in the tail of the two branches of the machine (states 5 and 7, and 6 and 8). Once those are merged, the resulting machine is clearly already minimal.

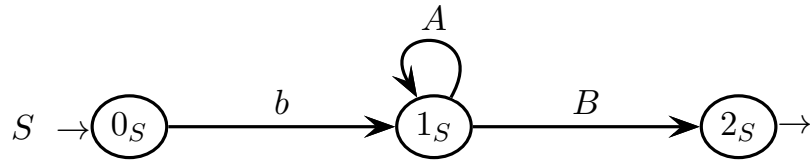
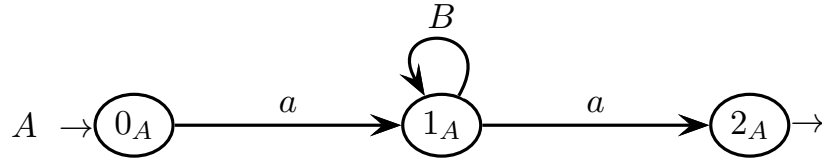
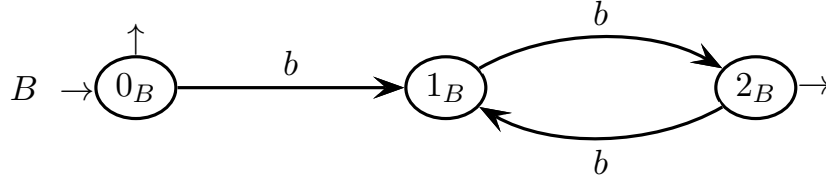
To prove this formally, one could proceed with Myhill-Nerode again, but now with fewer states to worry about.

- (f) The intersection $L(A_2) \cap L(R)$ is empty. Language $L(A_2)$ always starts with character b , so only the second part of the regular expression R is relevant, i.e., $(ba)^+(ab)^+$. Machine A_2 clearly recognizes exactly $(ba)^+$ as a prefix, like expression R , but then it expects a string of zero or more characters b , i.e., b^* , while expression R expects $(ab)^+$. The languages described by the two regular expressions b^* and $(ab)^+$ are completely disjoint, and as a result the common prefix is not enough for the two languages $L(A_2)$ and $L(R)$ to have strings in common.

This can also be seen by designing the intersection machine explicitly through the matrix construction. The resulting automaton (clearly non-clean) has no final state and is therefore unable to recognize any string.

2 Syntax analysis and parsing methodologies (10 points)

1. Consider the following grammar G , described by a machine net, over the two-letter terminal alphabet $\Sigma = \{a, b\}$ and the three-symbol nonterminal alphabet $V = \{A, B, S\}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

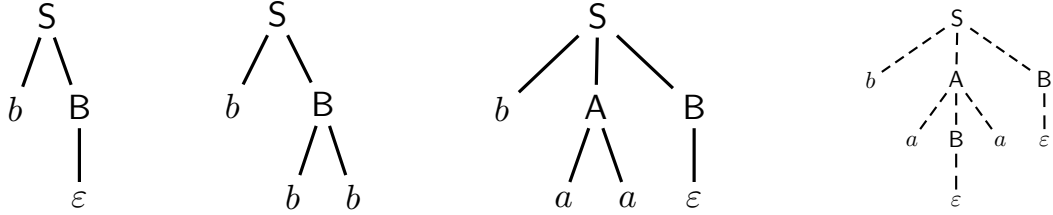
- List the strings of length 3 or less belonging to language $L(G)$ and draw their syntax trees.
- Build the complete pilot automaton and determine whether grammar G admits an ELR parser construction.
- Determine the guide sets on the machine net of grammar G and say whether there are conflicts that prevent the ELL parser construction.
- Independently of whether the machine net is ELL or not, write the recursive-descent parser procedure for the axiomatic machine S .
- Can you propose a modification to the machine net of grammar G that removes the conflicts without changing the language $L(G)$?

Solution

(a) Here is the list of the strings of length 3 or less belonging to language $L(G)$, three in total:

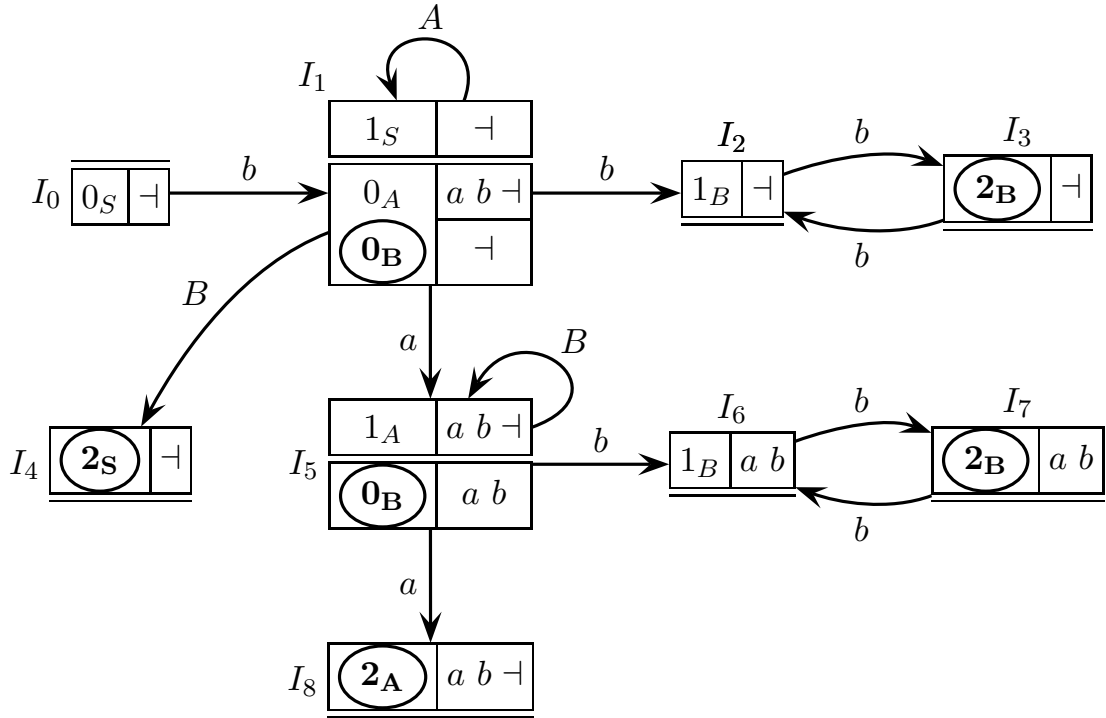
$$\begin{aligned}
 S &\xrightarrow{S \rightarrow bB} bB \xrightarrow{B \rightarrow \varepsilon} b\varepsilon = b \\
 S &\xrightarrow{S \rightarrow bB} bB \xrightarrow{B \rightarrow bb} bbb \\
 S &\xrightarrow{S \rightarrow bAB} bAB \xrightarrow{A \rightarrow aa} baaB \xrightarrow{B \rightarrow \varepsilon} baa\varepsilon = baa
 \end{aligned}$$

and their three syntax trees (solid):



Notice that string baa is highly ambiguous (actually infinitely ambiguous), since machines A and B could be equivalently written as EBNF rules as follows: $A \rightarrow aB^*a$ and $B \rightarrow \varepsilon \mid (bb)^+$. So the string can be generated by expanding to ε the nonterminal B in the rule of A as many times as desired. The solid tree of baa shown above is just the smallest one. Only for completeness, also the second smallest tree is shown above on the right, dashed.

(b) Here is the complete pilot of grammar G , with nine m-states, and the conflict analysis:

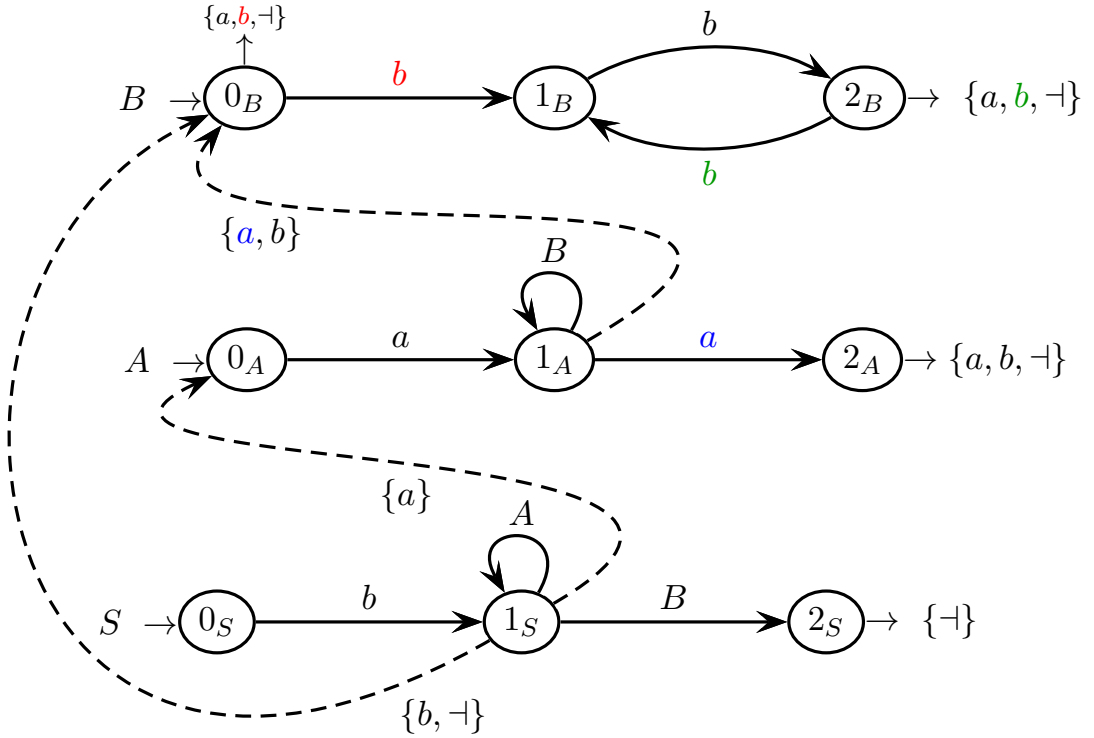


Here are all the conflicts in the pilot (three in total, only of type SR):

- m-state I_5 : shift-reduce conflict between $\langle \textcircled{0_B}, b \rangle$ and outgoing edge b
- m-state I_5 : shift-reduce conflict between $\langle \textcircled{0_B}, a \rangle$ and outgoing edge a
- m-state I_7 : shift-reduce conflict between $\langle \textcircled{2_B}, b \rangle$ and outgoing edge b

Grammar G is not ELR. This is expected if one notices that it is ambiguous, see point (a).

(c) Here are the guide sets on the call arcs and exit arrows, and the conflict analysis:



In the parser control-flow-graph (PCFG) we can see three conflicts (coloured): state 1_A has a conflict on a between the call arc to 0_B and the transition to 2_A , state 0_B has a conflict on b between the prospect set (i.e., the guide set on the exit arrow) and the transition to 1_B , and state 2_B has a conflict on b between the prospect set and the transition to 1_B .

As expected, the grammar is not ELL (1), as it is not ELR either. Notice however that the axiomatic machine S does not present any conflicts (it is deterministic).

(d) Here is the recursive-descent parser procedure of machine S , which is deterministic:

```

procedure  $S$  // standard version
if  $cc \in \{b\}$  then
   $cc := next$ 
  LOOP: if  $cc \in \{a\}$  then
    call  $A$ 
    goto  $LOOP$ 
  else if  $cc \in \{b, -\}$  then
    call  $B$ 
    if  $cc \in \{-\}$  then return
    else error
  else error
else error
end procedure

```

```

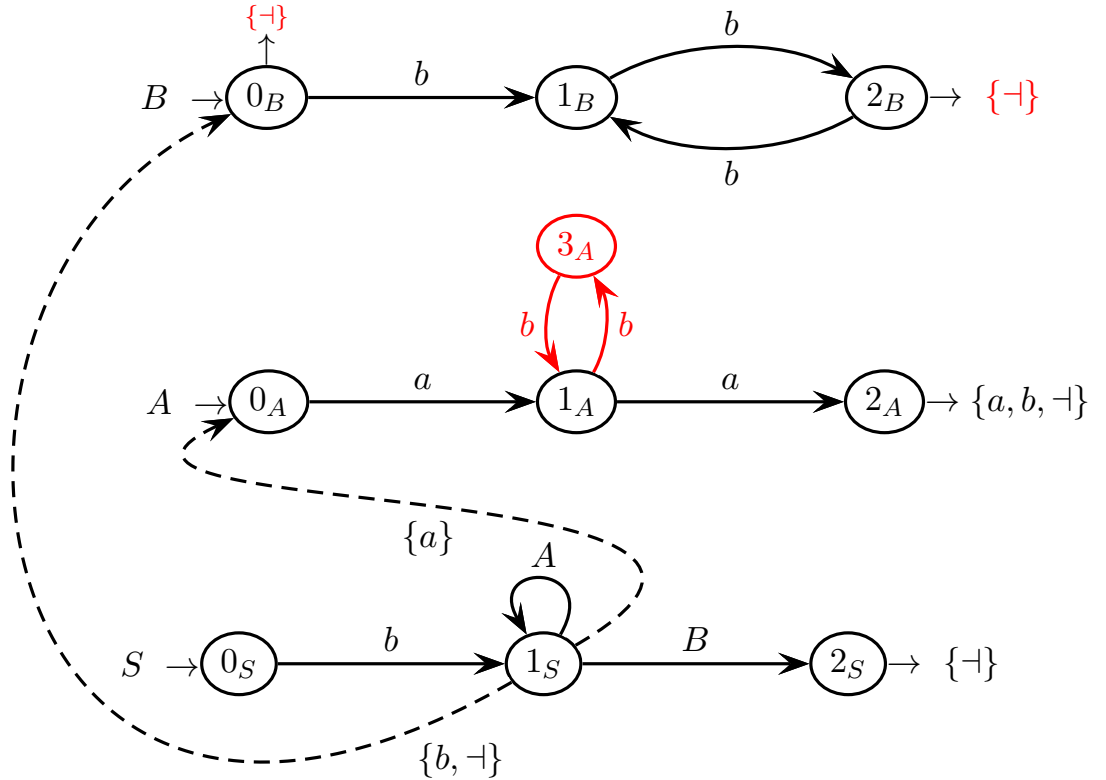
procedure  $S$  // optimized version
if  $cc \in \{b\}$  then
   $cc := next$ 
  while  $cc \in \{a\}$  do call  $A$ 
  if  $cc \in \{b, -\}$  then
    call  $B$ 
    if  $cc \in \{-\}$  then return
  error // all errors here
end procedure

```

For the other two machines (A and B) it is not possible to design deterministic recursive-descent syntactic procedures.

- (e) Grammar G is actually ambiguous. We can identify two forms of ambiguity. First, nonterminal B can be generated as many times as desired in state 1_A , but all those instances of B are nullable (see also point (a)). Second, any sequence of an even number of letters b can be generated by splitting it into two or more instances of B , e.g., $\underline{bb} \underline{bbbb}$ and $\underline{bbbb} \underline{bb}$, and even into infinitely many instances since B is nullable.

To remove all ambiguity, since nonterminal B simply generates the strings b^{2n} with $n \geq 0$, i.e., $(bb)^*$, it suffices to introduce a new state 3_A (non-final), and then the B self-loop on state 1_A can be replaced with an outgoing arc $1_A \xrightarrow{b} 3_A$ and an incoming arc $3_A \xrightarrow{b} 1_A$ (shown in red). The machine net becomes as follows (note that the guide set on the exit arrow of machine B has to be updated):



The three conflicts on states 1_A , 0_B and 2_B disappear, and no new ones appear. The new machine net is ELL, therefore it is also ELR (and consequently not ambiguous), and it does not have any conflicts of any kind. Variants may exist, of course.

Note that just making nonterminal B non-nullable, e.g., by turning state 0_B into non-final (and 1_S into final to preserve the language), removes the conflicts on 1_A and 0_B , but the conflict on 2_B remains, as it is related to the second form of ambiguity mentioned before.

A comprehensive analysis shows that language $L(G)$ is regular, as the machine net is not recursive. Thus a single deterministic automaton, with only terminal transitions, can recognize the language, and such an automaton would be of type ELL, without any conflicts. This however would be a complete redesign of grammar G , which totally changes its structure.

3 Grammar design – translation – semantic analysis (5 + 5 points)

1. Consider a language that describes linear combinations of vectors, with the following features:
 - Scalars and vectors are denoted by terminals `scal` and `vect`, respectively. Both terminals are unexpanded.
 - Scalars can be added, with operator `+`, and the result is of scalar type; the associativity of scalar addition is unspecified.
 - A linear combination consists of the addition of vectors, where each vector may be multiplied by a scalar value. The result of a linear combination is of vector type.
 - The vector addition operator is `+`, and it is right-associative.
 - The operator for scalar-vector multiplication is `*`, and the result is of vector type. The scalar value must be the left factor, and it may appear as a single scalar or a sum of scalars (between parentheses).Scalar-vector multiplication has higher precedence on both scalar addition and vector addition.
 - *Caution*: scalar-to-vector (or vector-to-scalar) addition is disallowed.
 - Vector subexpressions are allowed, at any depth, enclosed between parentheses.
 - A language phrase is a linear combination, as described above.

Sample linear combinations (symbols `sn` and `Vn` denote scalars and vectors, respectively):

`V1`
`V1 + V2`
`V1 + (V2 + V3)`
`V1 + s2 * V2`
`V1 + s1 * (V2 + (s2 + s3) * V3)`

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Write an EBNF non-ambiguous grammar (axiom `LComb`) that generates a language phrase.
- (b) Draw the syntax tree of the sample phrase:

`V1 + s1 * (V2 + (s2 + s3) * V3)`

- (c) Suppose now that scalars can be multiplied, with operator `*`; the result is of scalar type; scalar multiplication is *left-associative* and has higher precedence on scalar addition. Here is an example:

`V1 + (s1 + s2 * s3) * V2`

Modify the grammar of point (a), still EBNF and non-ambiguous, to include this generalization of the language.

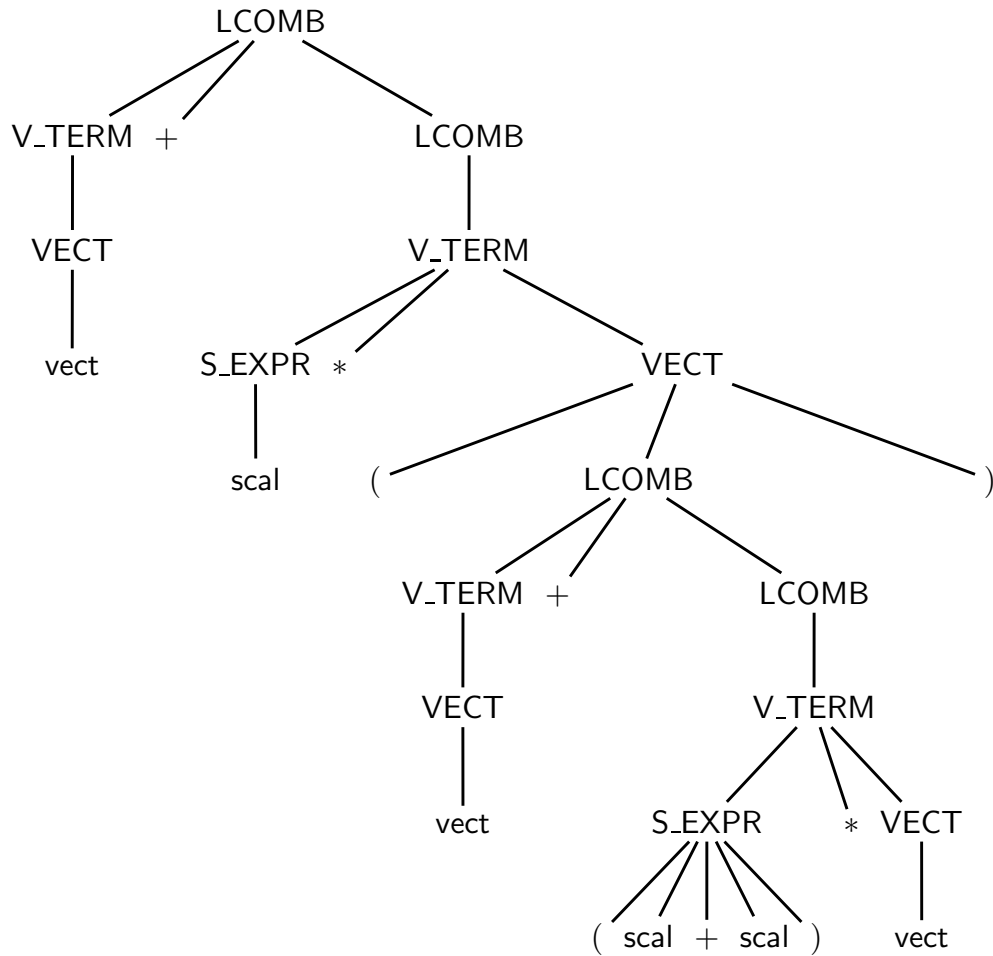
Solution

- (a) The key consists of distinguishing between scalar and vector subexpressions, as their associativity is modeled differently. Here is a viable grammar (axiom LCOMB):

$$\left\{ \begin{array}{l} \langle \text{LCOMB} \rangle \rightarrow \langle \text{V_TERM} \rangle \mid \langle \text{V_TERM} \rangle \text{ "+" } \langle \text{LCOMB} \rangle \quad // \text{ right-recursive} \\ \langle \text{V_TERM} \rangle \rightarrow [\langle \text{S_EXPR} \rangle \text{ "*" }] \langle \text{VECT} \rangle \\ \langle \text{S_EXPR} \rangle \rightarrow \text{scal} \mid \text{"(" scal ("+" scal)* ")} \quad // \text{ iterative} \\ \langle \text{VECT} \rangle \rightarrow \text{vect} \mid \text{"(" } \langle \text{LCOMB} \rangle \text{"}" } \end{array} \right.$$

The LCOMB rule is right-recursive for the right associativity of vector addition. The S_EXPR rule is simply iterative, since the associativity of scalar addition is unspecified.

- (b) Here is the translation tree of the sample valid string $V1 + s1 * (V2 + (s2 + s3) * V3)$:



- (c) The key consists of generalizing scalars, but the associativity and precedence have to be modeled properly, with left recursion and a new nonterminal S_TERM. Here is the modified grammar (axiom LCOMB):

$$\left\{ \begin{array}{l} \langle \text{LCOMB} \rangle \rightarrow \langle \text{V_TERM} \rangle \mid \langle \text{V_TERM} \rangle \text{ "+" } \langle \text{LCOMB} \rangle \quad // \text{ right-recursive} \\ \langle \text{V_TERM} \rangle \rightarrow [\langle \text{S_EXPR} \rangle \text{ "*" }] \langle \text{VECT} \rangle \\ \langle \text{S_EXPR} \rangle \rightarrow \text{scal} \mid \text{"(" } \langle \text{S_TERM} \rangle \text{ ("+" } \langle \text{S_TERM} \rangle \text{)* ")} \quad // \text{ iterative} \\ \langle \text{VECT} \rangle \rightarrow \text{vect} \mid \text{"(" } \langle \text{LCOMB} \rangle \text{"}" } \\ \langle \text{S_TERM} \rangle \rightarrow \text{scal} \mid \langle \text{S_TERM} \rangle \text{ "*" } \text{scal} \quad // \text{ left-recursive} \end{array} \right.$$

The modified or new rules are coloured in red. The S_TERM rule is left-recursive for the left associativity of scalar multiplication.

2. Consider the language of the nested lists, possibly empty, surrounded by open and closed parentheses, with elements a and b , as modeled by the source BNF grammar G^s below (axiom S):

$$G^s \left\{ \begin{array}{l} 1: S \rightarrow (L) \\ 2: L \rightarrow a L \\ 3: L \rightarrow b L \\ 4: L \rightarrow S L \\ 5: L \rightarrow \varepsilon \end{array} \right.$$

Answer the following questions (use the tables / trees / spaces on the next pages):

- (a) Consider a syntactic translation τ that, in each (sub)list, accumulates all the elements b after the elements a . For example:

$$\tau((a b b (b a) a)) = (a (a b) a b b)$$

Write a translation grammar G^τ (or a scheme) that realizes translation τ , with source grammar G^s . Draw the translation tree (or the separate source and target trees) of the sample translation τ below:

$$\tau((a b (b a) a)) = (a (a b) a b)$$

- (b) Is translation τ deterministic ELL ? Explain your answer.
- (c) Is the inverse translation τ^{-1} deterministic ELL ? Is it deterministic in general, i.e., does a deterministic pushdown translator exist for it ? Explain your answers.
- (d) Can we define a syntactic translation (with source grammar G^s) that, in each (sub)list, accumulates all the sublists at the sub(list) end ? For example:

$$(a b (b) a ()) \quad \text{is translated to} \quad (a b a (b) ())$$

Explain your answer.

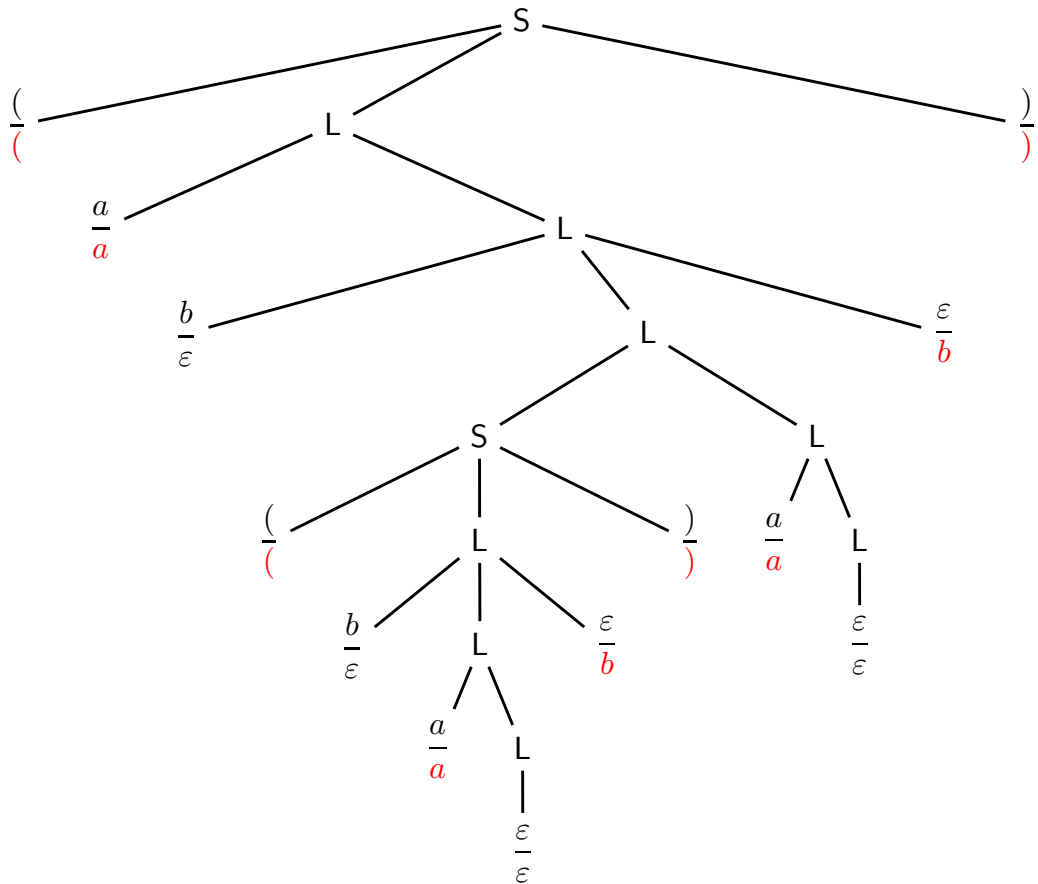
Solution

(a) Here is a viable translation grammar G^T (axiom S):

$$G^T \left\{ \begin{array}{l} 1: S \rightarrow \left(L \right) \\ 2: L \rightarrow \frac{a}{a} L \\ 3: L \rightarrow \frac{b}{\varepsilon} L \frac{\varepsilon}{b} \\ 4: L \rightarrow S L \\ 5: L \rightarrow \frac{\varepsilon}{\varepsilon} \end{array} \right.$$

All elements b are moved to the (sub)list end, after all elements a .

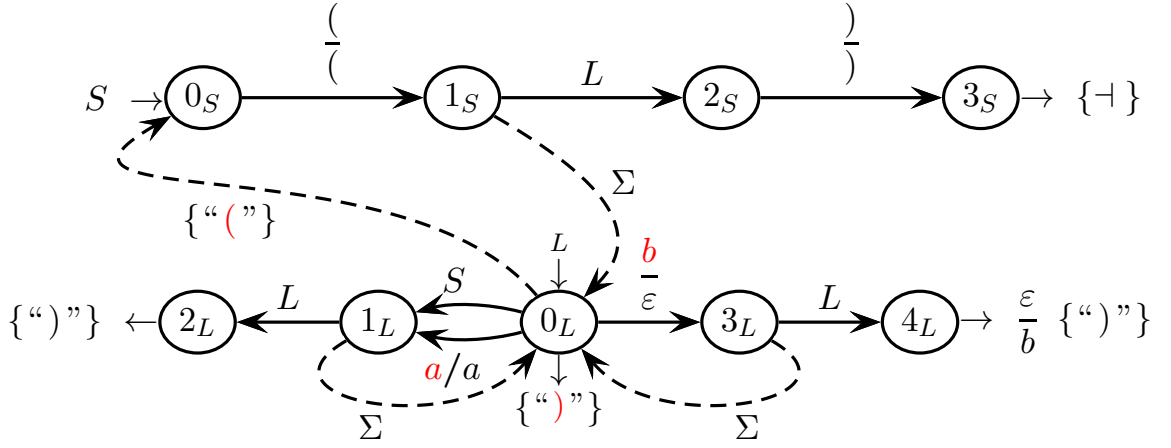
Here is the translation tree of the sample source string $(ab(ba)a) \xrightarrow{\tau} (a(ab)ab)$:



For clarity, the non-null characters of the destination (target) string are coloured in red.

(b) First of all, translation τ is one-valued, as it maps a string to a string. Then, since the rules of the source grammar G^s are BNF, the LL theory applies, i.e., separately looking at the guide sets of the alternative rules of nonterminal L . The LL guide sets of the four alternative rules 2, 3, 4 and 5 of L are orderly the singletons $\{a\}$, $\{b\}$, $\{“(”)\}$ and $\{“)”\}$, and are disjoint. Thus the source grammar G^s is LL, with $k = 1$.

Alternatively, one can notice that the source grammar is ELL, also with $k = 1$. The ELL net of translators is as follows (with $\Sigma = \{a, b, \text{“} \text{”}, \text{“} \text{”}\}$):



Notice that the exit arrow on state 4_L emits an output, as a terminal has to be emitted at the end of the path $0_L \rightarrow 3_L \rightarrow 4_L$ (see also the corresponding rule 3).

The unique bifurcation state is 0_L , yet there are no conflicts between the four guide sets (in red) on the four outgoing arcs (two terminal shift arcs, one call arc and one exit arrow). Thus in both cases (LL and ELL) a recursive-descent syntactic translator exists for translation τ , and the translation is deterministic.

- (c) A viable grammar $G^{\tau^{-1}}$ for the inverse translation τ^{-1} is as follows, obtained in the standard way by switching the source and destination grammars, i.e., by flipping the source / target terminal “fractions” (two fractions change in rule 3, the others do not):

$$G^{\tau^{-1}} \left\{ \begin{array}{l} 1: S \rightarrow \{ L \} \\ 2: L \rightarrow \frac{a}{a} L \\ 3: L \rightarrow \frac{\varepsilon}{b} L \frac{b}{\varepsilon} \\ 4: L \rightarrow S L \\ 5: L \rightarrow \frac{\varepsilon}{\varepsilon} \end{array} \right.$$

Grammar $G^{\tau^{-1}}$ is not ELL for any look-ahead depth $k \geq 1$, because its source rule 3: $L \rightarrow L b$ is left-recursive. Thus the translation τ^{-1} defined by means of $G^{\tau^{-1}}$ is not deterministic ELL, for any look-ahead depth $k \geq 1$.

One may wonder whether for translation τ^{-1} there is another grammar (not obtained in the standard way), possibly deterministic (ELL or ELR), or a deterministic pushdown translator (not obtained as a syntax analyzer of a grammar). Yet translation τ^{-1} is not one-valued, since a destination (target) string like (ab) could be remapped to both source strings (ba) or (ab) . Therefore, translation τ^{-1} is not deterministic in general, i.e., it does not have a deterministic pushdown translator.

- (d) We cannot do, since we should permute a whole subtree of the source grammar G^s , i.e., the source rule 4: $L \rightarrow S L$ should become $L \rightarrow L S$ in the destination (target) grammar, which means to change the syntactic structure of the destination (target) grammar w.r.t. the source grammar G^s , something that is not allowed in a syntactic translation model.

4 Bonus question (5 points)

1. Consider the following BNF grammar G (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow L \\ L \rightarrow A L \\ L \rightarrow \varepsilon \\ A \rightarrow a B \\ B \rightarrow b B \\ B \rightarrow \varepsilon \end{array} \right.$$

Note that it holds $L(G) = (ab^*)^*$, thus grammar G models a regular language.

We want to design an attribute grammar, based on the syntactic support G , to determine if a string of language $L(G)$ also belongs to the non-context-free language L' below:

$$L' = \{ (ab^n)^n \mid n \geq 0 \}$$

that is, if the number of letters b between each consecutive pair of letters a is the same as the number of letters a in the whole string. For example, string aa belongs to $L(G)$ but not to L' , whereas string $abba$ belongs to both languages, so it is acceptable.

The attribute grammar must produce a boolean attribute in the root node S of the syntax tree, which is true or false if the string is acceptable or not. We also want the attribute grammar to be suitable for integration in an ELL parser based on grammar G .

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Define what attributes are needed for this purpose, what their types are (left/right, data type), and to which nonterminals they are associated. One of the attributes is already given.
 - (b) Write the semantic functions for all the attributes defined at point (a).
 - (c) Prove that the devised attribute grammar has the one-sweep and L properties.
 - (d) Implement the corresponding integrated recursive-descent syntactic-semantic parsing procedure for nonterminal L .
-

list of attributes to be defined (the number of rows is not significant) – question (a)

<i>left/right</i>	<i>name</i>	<i>data type</i>	<i>nonterminal(s)</i>	<i>meaning</i>
left	r	boolean	S	correctness predicate

semantic functions – question (b)

#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow L_1$	
2:	$L_0 \rightarrow A_1 L_2$	
3:	$L_0 \rightarrow \varepsilon$	
4:	$A_0 \rightarrow a B_1$	
5:	$B_0 \rightarrow b B_1$	
6:	$B_0 \rightarrow \varepsilon$	

Solution

- (a) We can use the following four attributes, all of type left (synthesized):

<i>left/right</i>	<i>name</i>	<i>data type</i>	<i>nonterminal(s)</i>	<i>meaning</i>
left	r	boolean	S	correctness predicate
left	na	integer	L	number of letters a
left	nb	integer	A, B	number of consecutive letters b that follow a letter a
left	v	list of integers	L	list of numbers of consecutive letters b , i.e., $v = (i_1, i_2, \dots)$, where each element i is an integer

- (b) Here are the semantic functions. The operator “ $\cdot$ ” appends an element to a list, for instance $(i_1, i_2) \cdot i_3 = (i_1, i_2, i_3)$:

#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow L_1$	// formal version $r_0 := \text{if } (\forall i \in v_1 \ i = na_1) \text{ then True else False}$ // informal version $r_0 := \text{if all elements of } v_1 \text{ are equal to } na_1 \text{ then True else False}$
2:	$L_0 \rightarrow A_1 L_2$	$na_0 := na_2 + 1$ $v_0 := v_2 \cdot nb_1$ // length of $v_0 = \text{length of } v_2 + 1$
3:	$L_0 \rightarrow \varepsilon$	$na_0 := 0$ $v_0 := \text{empty list}$ // length of $v_0 = 0$
4:	$A_0 \rightarrow a B_1$	$nb_0 := nb_1$
5:	$B_0 \rightarrow b B_1$	$nb_0 := nb_1 + 1$
6:	$B_0 \rightarrow \varepsilon$	$nb_0 := 0$

The integer attribute na counts the number of factors ab^* in the whole string $(ab^*)^* = ab^*ab^* \dots ab^*$, the integer attribute nb counts the number of letters b in each such factor, and the list attribute v collects all the values taken by attribute nb , one value for each factor. Note that by construction the length of list v is equal to na in every node L of the tree (see the semantic functions and comments in rules 2 and 3), i.e., $v = (i_1, i_2, \dots, i_{na})$, where each element i is an integer (or $v = \text{empty list}$ if $na = 0$). Therefore (in the root node S) the boolean attribute r is set to true if and only if all the integer elements i contained in v are equal to na , i.e., $i_1 = i_2 = \dots = i_{na} = na$. This is exactly the test that the syntactic support G alone cannot perform and instead requires an attribute grammar like the one above.

- (c) The attribute grammar has only left attributes, i.e., it is purely synthesized, thus it is trivially one-sweep. Moreover, the sibling graphs of all the rules have no arcs, since all the attributes are left and there are no dependencies between child nodes. Thus the attribute grammar is even of type L. Since grammar G is ELL (with $k = 1$), for the attribute grammar it is possible to design an integrated syntactic-semantic recursive-descent analyzer.

- (d) Here is the encoding of the integrated syntactic-semantic recursive-descent procedure L , along with the headers of the others. All the procedures only have output parameters (since all the attributes are synthesized):

```

procedure  $S$  (out:  $r_0$ )                                // header only
procedure  $A$  (out:  $nb_0$ )                                // header only
procedure  $B$  (out:  $nb_0$ )                                // header only


---


procedure  $L$  (out:  $na_0, v_0$ )                            // header and body
var  $na_2, v_2, nb_1$                                      // local variables (attributes of child nodes)
if  $cc \in \{a\}$  then                                     // rule  $L \rightarrow AL$ 
|   call  $A(nb_1)$                                        // counts consecutive letters  $b$  and returns  $nb$ 
|   if  $cc \in \{a, \vdash\}$  then
|   |   call  $L(na_2, v_2)$                                // recursively updates  $na$  and  $v$ 
|   |   if  $cc \in \{\vdash\}$  then
|   |   |    $na_0 := na_2 + 1$                              // increments number of letters  $a$ 
|   |   |    $v_0 := v_2 \cdot nb_1$                        // appends element  $nb$  to list  $v$ 
|   |   |   return                                     // returns  $na$  and  $v$ 
|   |   else error
|   else error
else if  $cc \in \{\vdash\}$  then                             // rule  $L \rightarrow \varepsilon$ 
|    $na_0 := 0$ 
|    $v_0 := \emptyset$ 
|   return                                             // returns  $na$  and  $v$  (initial values)
else error
end procedure

```

More compactly (all headers but L are omitted):

```

procedure  $L$  (out:  $na_0, v_0$ )                            // header and body
var  $na_2, v_2, nb_1$                                      // local variables (attributes of child nodes)
if  $cc \in \{a\}$  then                                     // rule  $L \rightarrow AL$ 
|   call  $A(nb_1)$                                        // counts consecutive letters  $b$  and returns  $nb$ 
|   if  $cc \in \{a, \vdash\}$  then
|   |   call  $L(na_2, v_2)$                                // recursively updates  $na$  and  $v$ 
|   |   if  $cc \in \{\vdash\}$  then
|   |   |    $na_0 := na_2 + 1$                              // increments number of letters  $a$ 
|   |   |    $v_0 := v_2 \cdot nb_1$                        // appends element  $nb$  to list  $v$ 
|   |   |   return                                     // returns  $na$  and  $v$ 
|   else if  $cc \in \{\vdash\}$  then                             // rule  $L \rightarrow \varepsilon$ 
|   |    $na_0 := 0$ 
|   |    $v_0 := \emptyset$ 
|   |   return                                             // returns  $na$  and  $v$  (initial values)
|   error                                             // all syntactic errors here
end procedure

```

Both procedure versions have arguments and local variables in one-to-one correspondence with the argument instances in the semantic functions. Some further simplification, purely programmatic, is possible, e.g., by sparing some local variables. This is not relevant here.